

Python vs. R Implementation Comparison

K-Nearest Neighbors (KNN) Classification

Project Summary

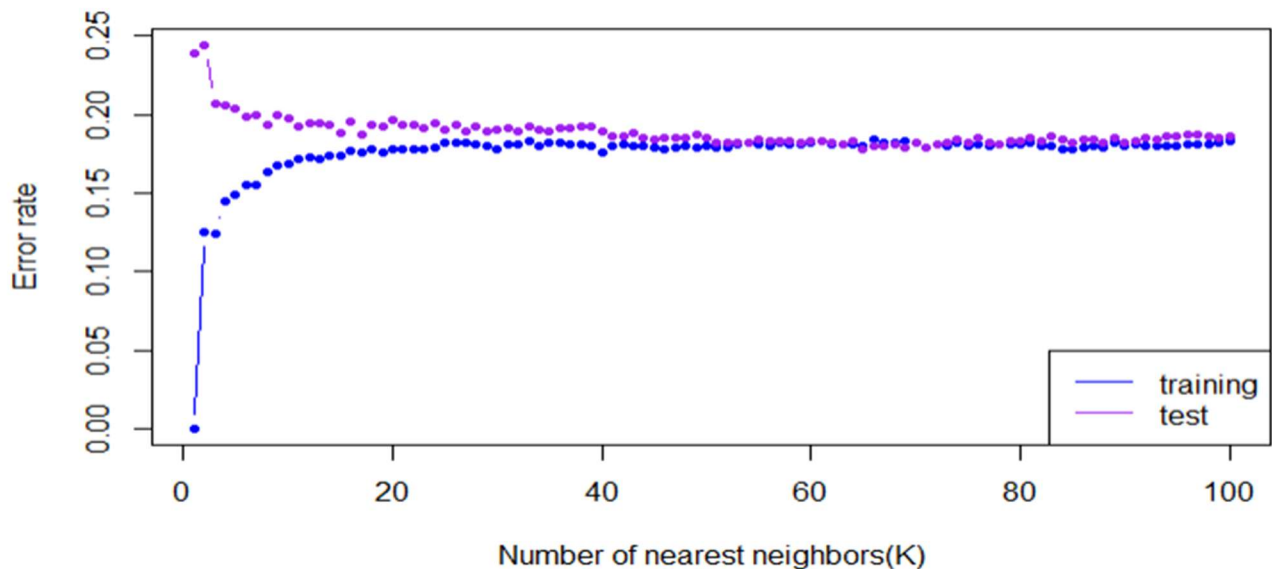
This document compares the implementation and performance of the K-Nearest Neighbors (KNN) classification algorithm using both Python and R. The comparison focuses on model performance, optimal parameter selection, and decision boundary analysis based on the same training and testing datasets.

Training and Test Error Analysis

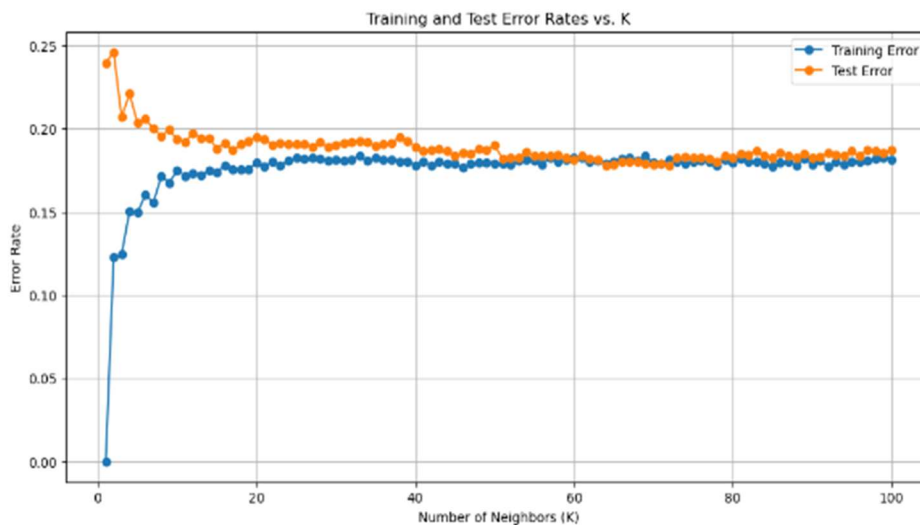
The K-Nearest Neighbors (KNN) classifier was evaluated using values of **K** ranging from **1 to 100** in both Python and R. The figures below illustrate the relationship between the number of neighbors and the corresponding training and test error rates.

R Implementation

Plot of training and test error rates against K



Python Implementation



Observations

Both the Python and R implementations exhibited similar performance trends. The training error was lowest when $K = 1$ and gradually increased as the number of neighbors increased. In contrast, the test error initially decreased, reached a minimum near the optimal K value, and then remained relatively stable for larger values of K .

The error curves demonstrate the expected behavior of the KNN algorithm. Smaller values of K produce more flexible models that may overfit the training data, whereas larger values of K generate smoother decision boundaries with improved generalization.

Although the overall trends were nearly identical, the R implementation showed slightly greater variation in test error for smaller K values. These minor differences are likely attributable to implementation details, numerical precision, or package-specific computations rather than differences in the underlying KNN algorithm.

Overall, both implementations produced consistent results and demonstrated comparable classification performance.

Optimal K Selection

The optimal value of K was determined by identifying the model with the lowest test error.

Implementation	Optimal K	Training Error	Test Error
Python	64	0.1790	0.1780
R	65	0.18805	0.1785

Discussion

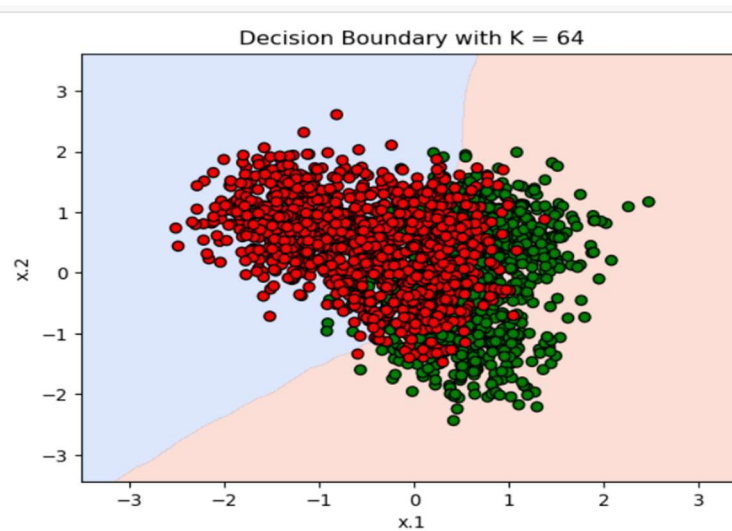
Both implementations identified nearly identical optimal values of K , differing by only one neighbor. This small difference resulted in very similar training and test error rates, indicating that both Python and R produced consistent classification performance on the same dataset.

The slight variation in the optimal K value is expected and may arise from differences in numerical precision, package implementation, or internal algorithm behavior. Despite these differences, both models demonstrated comparable predictive accuracy and selected a neighborhood size that balanced model complexity with generalization performance.

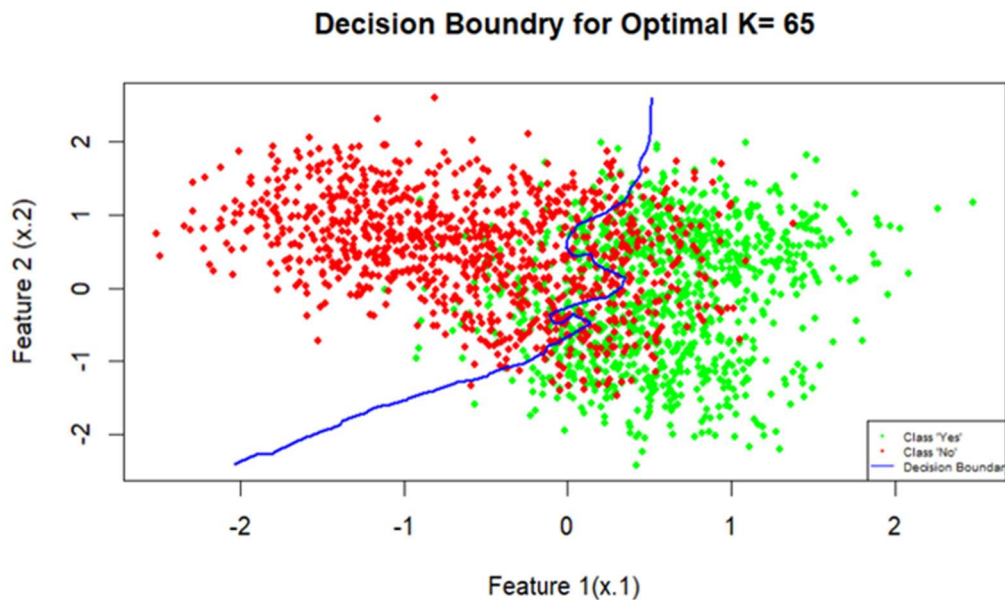
Decision Boundary Analysis

The decision boundaries generated using the optimal K values illustrate how the KNN classifier separates the two classes based on the training data.

Python Implementation (K = 64)



R Implementation (K = 65)



Observations

Both implementations produced decision boundaries that effectively separated the two classes while capturing the overall structure of the data. The Python implementation generated a relatively smooth and generalized decision boundary, whereas the R implementation produced a slightly more irregular boundary. These differences are minor and are likely due to implementation-specific details rather than differences in the KNN algorithm itself.

Some overlap between the classes resulted in a small number of expected misclassifications, particularly in regions where the data points were closely intermixed. Overall, both decision boundaries appropriately reflected the underlying data distribution without showing signs of excessive overfitting.

Conclusion

This project demonstrated that both Python and R provide reliable implementations of the K-Nearest Neighbors (KNN) classification algorithm. Although the optimal value of K differed slightly between the two implementations (Python: **K = 64**, R: **K = 65**), both achieved nearly identical training and test error rates.

The error rate curves and decision boundary visualizations confirmed the expected behavior of the KNN algorithm. As the number of neighbors increased, the model became more generalized, illustrating the trade-off between model complexity and generalization. Overall, both implementations produced consistent classification performance and validated the effectiveness of KNN for this binary classification problem.